COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 / Data Structures and Algorithms | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | 7/21/26 | **DATE SUBMITTED:** | 7/21/26 |
| **NAME:** | Payas, John Paul S. | **INSTRUCTOR:** | Sir. Jimlord M Quejado |

**ACTIVITY NO. 2.1** **Arrays, Pointers and Dynamic Memory Allocation**

## 1. PROCEDURE OUTPUT

| Screenshot | **Code:** |
|---|---|

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    Student& operator=(const Student &copyStudent) {
        if (this != &copyStudent) {
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        return *this;
    }

    void printDetails() {
        std::cout << studentName << " " << studentAge << std::endl;
    }
};

int main() {

    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

  return 0;

}
```

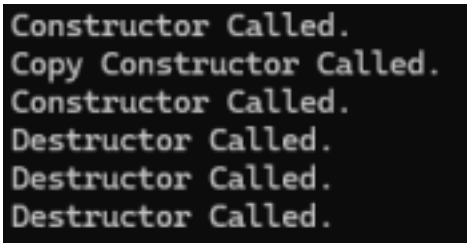
| | |
|---|---|
| | **Output:**<br>  |
| Observation | It creates three Student objects. The parameterized constructor initialized student1, the copy constructor duplicated student1 into student2, and the default constructor initialized student3. The assignment statement copied the values from student2 to student3. When the program finished, the destructors were automatically called, demonstrating the object creation and destruction process. |

**Table 2-1. Initial Driver Program**

| | |
|---|---|
| Screenshot | **Code:**<br>`int main() {`<br>`const int j = 5;`<br>`Student studentList[j];`<br>`std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};`<br>`int ageList[j] = {15,16,18,19,16};`<br>`return 0;`<br>`}` |

COMPUTER ENGINEERING
QUEZON CITY

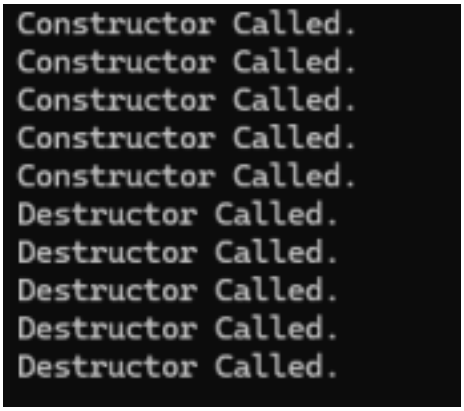
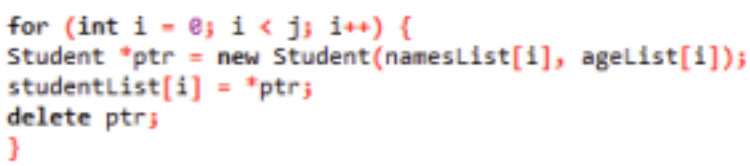
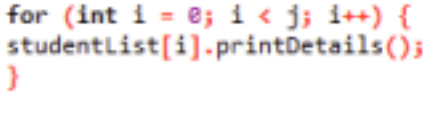
| | |
|---|---|
| | **Output:**<br>  <br> |
| Observation | It creates a program that successfully creates a static array containing five Student objects. Each object was initialized using the default constructor. The namesList and ageList arrays stored the corresponding student information. |

**Table 2-2. Modified Driver Program with Student Lists**

| | |
|---|---|
| Loop A |  <br> |
| Observation | On loop A it allocates a Student object in the heap using the new operator. Each object is initialized with the corresponding name and age from the namesList and ageList arrays. The object is then copied into the studentList array, and the allocated memory is released using the delete operator to prevent memory leaks. |
| Loop B |  <br> |

| | |
|---|---|
| Observation | On loop B it traverses the studentList array and calls the printDetails() function for each object. It displays the name and age of every student, confirming that the data was correctly |

COMPUTER ENGINEERING
QUEZON CITY

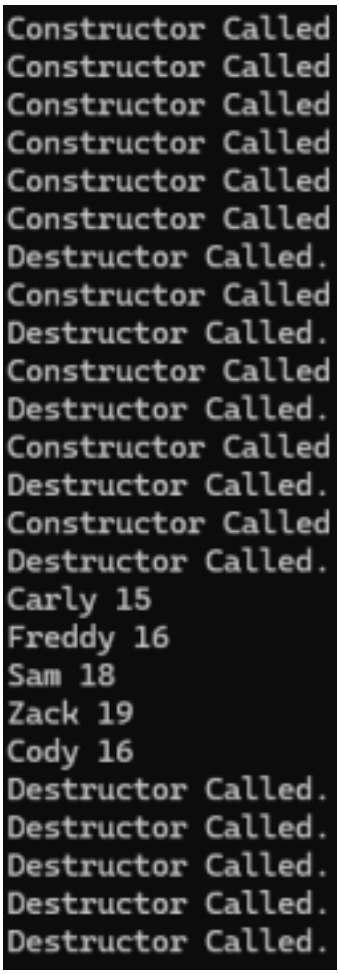
| | |
|---|---|
| Output |  |
| Observation | The output successfully demonstrates both static and dynamic memory allocation. constructors are called when objects are created, and the destructors are called when dynamically allocated objects are deleted and when local objects go out of scope. The student information is also displayed correctly, indicating that the dynamic allocation, object copying, and memory deallocation all work as intended. |

Table 2-3. Final Driver Program

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

### ILO C:

### Problem 1-4

```
#include <iostream>
#include <string>

class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    Fruit(std::string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    ~Fruit() {
    }

    Fruit(const Fruit &copy) {
        name = copy.name;
        price = copy.price;
        quantity = copy.quantity;
    }

    Fruit& operator=(const Fruit &copy) {
        if (this != &copy) {
            name = copy.name;
            price = copy.price;
            quantity = copy.quantity;
        }
        return *this;
    }

    double calculateSum() {
        return price * quantity;
    }

    void display() {
        std::cout << "Fruit: " << name
                  << " | Price: PHP " << price
                  << " | Quantity: " << quantity
                  << " | Total: PHP " << calculateSum()
                  << std::endl;
    }
};

class Vegetable {
private:
    std::string name;
```

```
class Vegetable {
    double price;
    int quantity;

public:
    Vegetable(std::string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    ~Vegetable() {
    }

    Vegetable(const Vegetable &copy) {
        name = copy.name;
        price = copy.price;
        quantity = copy.quantity;
    }

    Vegetable& operator=(const Vegetable &copy) {
        if (this != &copy) {
            name = copy.name;
            price = copy.price;
            quantity = copy.quantity;
        }
        return *this;
    }

    double calculateSum() {
        return price * quantity;
    }

    void display() {
        std::cout << "Vegetable: " << name
                  << " | Price: PHP " << price
                  << " | Quantity: " << quantity
                  << " | Total: PHP " << calculateSum()
                  << std::endl;
    }
};

double TotalSum(Fruit *fruits[], int fruitCount,
                Vegetable *vegetables[], int vegetableCount) {

    double total = 0;

    for (int i = 0; i < fruitCount; i++) {
        if (fruits[i] != NULL)
```

```
            total += fruits[i]->calculateSum();
    }

    for (int i = 0; i < vegetableCount; i++) {
        if (vegetables[i] != NULL)
            total += vegetables[i]->calculateSum();
    }

    return total;
}

int main() {

    Fruit *fruits[2];

    fruits[0] = new Fruit("Apple", 10, 7);
    fruits[1] = new Fruit("Banana", 10, 8);

    Vegetable *vegetables[2];

    vegetables[0] = new Vegetable("Broccoli", 60, 12);
    vegetables[1] = new Vegetable("Lettuce", 50, 10);

    std::cout << "Jenna's Grocery List\n\n";

    fruits[0]->display();
    fruits[1]->display();
    vegetables[0]->display();
    vegetables[1]->display();

    std::cout << "\nTotal Amount: PHP "
            << TotalSum(fruits, 2, vegetables, 2)
            << std::endl;

    delete vegetables[1];
    vegetables[1] = NULL;

    std::cout << "\nAfter deleting Lettuce:\n\n";

    fruits[0]->display();
    fruits[1]->display();
    vegetables[0]->display();

    std::cout << "\nUpdated Total Amount: PHP "
            << TotalSum(fruits, 2, vegetables, 2)
            << std::endl;

    delete fruits[0];
```

```
    delete fruits[0];
    delete fruits[1];
    delete vegetables[0];

    return 0;
}
```

**Problem 5:**

The program uses separate Fruit and Vegetable classes to organize grocery items. Each class contains a constructor, destructor, copy constructor, and copy assignment operator to manage object creation and copying. The grocery items are dynamically allocated using pointers and stored in arrays. The TotalSum() function computes the total cost of all items by multiplying each item's price by its quantity. After deleting the Lettuce object using the delete operator, the program updates the array by assigning NULL to the pointer and recalculates the total cost. This demonstrates proper use of dynamic memory allocation and memory deallocation while preventing memory leaks.

**Output:**

```
Jenna's Grocery List

Fruit: Apple | Price: PHP 10 | Quantity: 7 | Total: PHP 70
Fruit: Banana | Price: PHP 10 | Quantity: 8 | Total: PHP 80
Vegetable: Broccoli | Price: PHP 60 | Quantity: 12 | Total: PHP 720
Vegetable: Lettuce | Price: PHP 50 | Quantity: 10 | Total: PHP 500

Total Amount: PHP 1370

After deleting Lettuce:

Fruit: Apple | Price: PHP 10 | Quantity: 7 | Total: PHP 70
Fruit: Banana | Price: PHP 10 | Quantity: 8 | Total: PHP 80
Vegetable: Broccoli | Price: PHP 60 | Quantity: 12 | Total: PHP 720

Updated Total Amount: PHP 870
```

**Analysis of the Supplementary Activity:**

The supplementary activity applied the concepts learned in a practical scenario by creating Fruit and Vegetable classes for a grocery list. Using dynamic memory allocation, pointers, and arrays allowed the program to manage grocery items efficiently. Implementing the TotalSum() function demonstrated how object data can be processed to solve a real-world problem, while deleting the lettuce object emphasized the importance of releasing dynamically allocated memory to prevent memory leaks. This activity reinforced the use of object-oriented programming together with dynamic memory management.

**Summary of Lessons Learned:**

This enhanced my understanding of arrays, pointers, and dynamic memory allocation. I learned the difference between static and dynamic memory allocation, how pointers reference memory addresses, and how the new and delete operators are used to allocate and deallocate memory. I also learned the importance of constructors, destructors, copy constructors, and copy assignment operators in managing objects and preventing memory-related problems such as memory leaks

**Concluding statement / Feedback:**

I do not think I performed very well in this activity because I found many of the concepts difficult to understand, especially pointers and dynamic memory allocation. I relied heavily on the examples provided and needed additional guidance to complete the tasks correctly. My main areas for improvement are writing C++ programs more independently, strengthening my understanding of pointers and memory management, and improving my debugging skills.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Quillbot, ChatGpt**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correctionCode assistance, and Explanation of the concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **It just fix my grammar and make it better but it did not replace the original thinking It's my own work and ChatGpt for like how to use that function properly**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY